

Unchecked call return value

Samuel Lechner Philipp Gorke

November 2, 2023

Similar to a distributed system, the block-chain and smart-contract environment relies on allowing not only communication between users-to-contracts, but also contracts-to-contracts. However, this makes the usage of contracts using this communication even more prone to errors and weaknesses. There are various ways in calling other contracts in the solidity language. Some of these functions bubble exceptions up and make the caller contract fail, whilst some just ignore the exception and return the success in a boolean value. This can lead to situations in which caller contracts resume execution unintended. Therefore, it is of the utmost importance for the developer to be aware of the various calling methods and their corresponding outcome options. Because there may be situations where the caller contract is uninterested in the call's result, an unchecked call return value should be regarded as a weakness, but not necessarily a vulnerability. However, it's good practise to always check return values where ever it's possible.

The validation of a method call's return value is not only important in the scope of Smart Contracts but in every software development process. Ensuring that the program is in the expected and desired state after a method invocation will enhance control flow and drastically reduce the likelihood of unwanted behavior.

1 Calls in Solidity

There are several ways to call Smart Contracts. Generally, a distinction can be made between High- and Low Level Calls.

1.1 High Level Calls

To interact with contracts both from outside the blockchain and for contract-to-contract interaction the Application binary interface (ABI) is required, which is a standardized scheme that defines how to call functions in that contract and get data back. These are called High Level Calls. [1]

Examples

```
1 contract A {
2     function f(uint _num) public returns (uint) {
3         return _num + 1;
4     }
5 }
6
7 contract B {
8     A a;
9     uint result;
10    function g(address _adr, uint _num) public {
11        a = A(_adr);
12        result = a.f(_num);
13    }
14 }
```

```
1 contract A {
2     function f() external payable { ... }
3 }
4
5 contract B {
6     A a = new A();
7     function g() public {
8         a.f{ value :1 ether }();
9     }
10 }
```

1.1.1 Send Ether to another contract

Ether sent to the address of a contract is processed by the **receive** function if it exists or the **fallback** function, if it's payable and there is no **receive** function. In any other case Ether is refused and the transaction reverted. However, there exist special methods [4] for injecting Ether into a contract, but that's beyond the scope of this discussion.

Examples

```
1 address payable to;
2 to.transfer(1 ether) ;
```

```

1 address payable to;
2 bool status = to.send(1 ether);
3 require(status);
4 // further code

```

1.2 Low Level Calls

However, if the caller contract does not have the callee's ABI, there are still option to interact with it. Solidity code cannot be executed by the EVM directly but have to be compiled to low-level machine instructions called **opcodes** that are encoded to bytecode. Low Level calls have a corresponding opcode and will either return 1 for **success** or 0 for **fail**. There are essentially three call methods.

opcode	name	parameters
0xF1	CALL	gas, to, val, input, output
0xF4	DELEGATECALL	gas, to, input, output
0xFA	STATICCALL	gas, to, input, output

CALL: Execute code of another contract.

DELEGATECALL Executing code from another contract, using the caller's state, allows dynamic code loading from a different address at runtime, with storage, current address, and balance referring to the caller's context, while only the code is fetched from the called address.

STATICCALL is similar to **CALL** but fails if contract tries to modify the state in any way.

```

1 address to;
2 bool status ;
3 bytes memory input;
4 bytes memory output;
5 // send all remaining gas
6 (status, output) = to.staticcall(input);

```

```

1 address to;
2 bool status ;
3 bytes memory input;
4 bytes memory output;
5 // send only 2300 gas for instance
6 (status, output) = to.delegatecall{gas: 2300}(input);

```

```

1 address to;
2 bool status ;
3 bytes memory input;
4 bytes memory output;
5 // send only 2300 gas and 1 ether
6 (status, output) = to.call{value: 1 ether, gas: 2300}(input);

```

2 Weakness specification

In this chapter we want to provide further information about the weaknesses. The focus will be specifically on Low-Level-Calls, especially on the functionality of `CALL`. However, all function calls, such as: `DELEGATECALL`, `SEND` or `STATICCALL` all translate to `CALL` in EVM-code.

2.1 Weaknesses of `CALL` function and its return value

If a smart contract calls another contract's method using `CALL`, a boolean value, as well as an optional value is returned. In the case that the function in the called contract fails or throws an exception, the caller contract's execution is still resumed. This leads to unexpected behaviors and can be exploited. In case of failure in the called contract, after using `CALL` or any equivalent function, the boolean-return value is `false` and therefore has to be validated after every call to a foreign contract. The weakness itself is the poor design and visibility of calling another contract. Other programming languages would fail, throw an exception or a specific error code. Furthermore, when calling a method, which does not exist, the `fallback()` method is triggered automatically which could also lead to unexpected behavior. [2]

2.2 Weaknesses of other Low-Level Calls

Since `DELEGATECALL` calls a method in another contract with the callers state, it's very prone to vulnerabilities. Storage, `msg.value` or the sender still refer to the contract that is calling the method and can therefore be changed and exploited easily. Further information is provided in [3]. In addition, the same weaknesses that apply for `CALL` also apply to `DELEGATECALL`.

3 Consequences of Exploit

The consequences of the exploit are versatile and dependant on the program logic of the contract. Without checking the return value the state of the contract could be updated incorrectly and lead to unwanted behaviors. An attacker could purposely (and repeatedly) make the call to the foreign contract fail, while still updating the caller contract's state. With this method, the attacker could abuse other weaknesses to further exploit the contract, such as a reentrancy attack [4]. In certain scenarios this could lead to loss of funds, either from other users who deposit ether on the contract, or even the contracts balance itself. In addition, when bringing the contract in an unwanted state, this could lead to various opportunities dependant on the program flow. In the worst case, even being able to perform `SELFDESTRUCT`, breaking the contract's functionality or making the contract fail when called.

4 Countermeasure

The most important countermeasure against the weakness is validation. After each execution of the `CALL` function to a foreign contract, the returned boolean value has to be checked and validated. This can be done with simple code, such as:

```
1 (bool success) = _addr.call{value: msg.value}
2 require(success, "Transfer failed")
```

or a simple `if` check. Of course not all unchecked calls to another contract lead to a vulnerability. Each case is distinct and has to be examined separately. For simple deposits of `msg.value` using the `TRANSFER` function instead is recommended. Furthermore, minimizing calls to external contracts is recommended, to decrease the possibility of an actual weakness.

5 Example contracts

In the following there are a few example contracts demonstrating the weakness.

Example 1

Tantikul [5] proposes a very simple, but meaningful example in his work:

```
1 msg.sender.send(amount);
2 wallet[msg.sender] -= amount ;
```

In this example the caller sends an amount of ether to a foreign contract. A mapping variable: `wallet`, that keeps track of the amount of ether each user has stored on the contract, is updated right after. If the `.send(amount)` call would fail, the `wallet` mapping would still be updated, causing an incorrect state and the foreign contract not receiving any ether.

Example 2

Consider the following example published on *hackingdistributed.com* [6], which is quite similar to the previous one:

```
1 if (gameHasEnded && !( prizePaidOut ) ) {
2     winner.send(1000);
3     prizePaidOut = True;
4 }
```

This piece of code might be found in a smart contract that implements a board game. Again, the problem here is that the boolean variable `prizePaidOut` is set to `True` without checking whether the sent method in the line before finished properly or failed (for whatever reason) and returned `False`.

References

- [1] The Solidity Authors. Contract abi specification, 2023.
- [2] Puneet Gill, Indrani Ray, Alireza Lotfi Takami, and Mahesh Tripunitara. Finding unchecked low-level calls with zero false positives and negatives in ethereum smart contracts. In Guy-Vincent Jourdan, Laurent Mounier, Carlisle Adams, Florence Sèdes, and Joaquin Garcia-Alfaro, editors, *Foundations and Practice of Security*, pages 305–321, Cham, 2023. Springer Nature Switzerland.
- [3] Fabio Gritti, Nicola Ruaro, Robert McLaughlin, Priyanka Bose, Dipanjan Das, Ilya Grishchenko, Christopher Kruegel, and Giovanni Vigna. Confusum contractum: Confused deputy vulnerabilities in ethereum smart contracts. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1793–1810, Anaheim, CA, August 2023. USENIX Association.
- [4] Nipun Sharma, Swati Sharma, and Anupama Sindgi. Solidity smart contract vulnerabilities, attack scenarios, and mitigation—a survey. In Sandeep Kumar, Saroj Hiranwal, S.D. Purohit, and Mukesh Prasad, editors, *Proceedings of International Conference on Communication and Computational Technologies*, pages 901–910, Singapore, 2023. Springer Nature Singapore.
- [5] Phitchayaphong Tantikul and Sudsanguan Ngamsuriyaroj. Exploring vulnerabilities in solidity smart contract. In *ICISSP*, pages 317–324, 2020.
- [6] Zikai Alex Wen and Andrew Miller. Scanning live ethereum contracts for the ”unchecked-send” bug, 2016.